Volume 06 - DNA Operating System Specifications

Version v26 draft

spec-06-dna-01-dna-kernel-specification.md

DNA Kernel Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-001

Related Blueprint Documents

- MAL-V03-B03-06-001
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for DNA kernel orchestration, package authority, lifecycle ownership, and runtime integration in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for DNA kernel orchestration, package authority, lifecycle ownership, and runtime integration.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.

- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.
- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.

- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNAKernelRequested
- DNAKernelValidated
- DNAKernelRejected
- DNAKernelMounted
- DNAKernelStateChanged
- DNAKernelFailed
- DNAKernelRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.
- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-001
- MAL-V03-B03-06-001
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-02-dna-package-engine-specification.md

DNA Package Engine Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-002

Related Blueprint Documents

- MAL-V03-B03-06-002
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for package parsing, validation, loading, execution preparation, and package state control in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for package parsing, validation, loading, execution preparation, and package state control.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNAPackageEngineRequested
- DNAPackageEngineValidated
- DNAPackageEngineRejected
- DNAPackageEngineMounted
- DNAPackageEngineStateChanged
- DNAPackageEngineFailed
- DNAPackageEngineRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-002
- MAL-V03-B03-06-002
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-03-mdp-import-export-specification.md

MDP Import Export Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-003

Related Blueprint Documents

- MAL-V03-B03-06-010
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for Mariam DNA Package import, export, metadata, compatibility, and transport packaging in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for Mariam DNA Package import, export, metadata, compatibility, and transport packaging.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- MDPImportExportRequested
- MDPImportExportValidated
- MDPImportExportRejected
- MDPImportExportMounted
- MDPImportExportStateChanged
- MDPImportExportFailed
- MDPImportExportRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-003
- MAL-V03-B03-06-010
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-04-zip-git-archive-adapters-specification.md

ZIP Git Archive Adapters Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-004

Related Blueprint Documents

- MAL-V03-B03-06-010
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for ZIP, Git, and archive adapters for source acquisition, integrity checks, and package materialization in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for ZIP, Git, and archive adapters for source acquisition, integrity checks, and package materialization.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- ZIPGitArchiveAdaptersRequested
- ZIPGitArchiveAdaptersValidated
- ZIPGitArchiveAdaptersRejected
- ZIPGitArchiveAdaptersMounted
- ZIPGitArchiveAdaptersStateChanged
- ZIPGitArchiveAdaptersFailed
- ZIPGitArchiveAdaptersRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-004
- MAL-V03-B03-06-010
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-05-package-runtime-object-specification.md

Package Runtime Object Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-005

Related Blueprint Documents

- MAL-V03-B03-06-003
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime object representation for mounted DNA packages, handles, identity, state, and authority in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime object representation for mounted DNA packages, handles, identity, state, and authority.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- PackageRuntimeObjectRequested
- PackageRuntimeObjectValidated
- PackageRuntimeObjectRejected
- PackageRuntimeObjectMounted
- PackageRuntimeObjectStateChanged
- PackageRuntimeObjectFailed
- PackageRuntimeObjectRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-005
- MAL-V03-B03-06-003
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-06-package-graph-specification.md

Package Graph Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-006

Related Blueprint Documents

- MAL-V03-B03-06-004
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for package graph nodes, dependency edges, composition, lineage, conflict tracking, and graph queries in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for package graph nodes, dependency edges, composition, lineage, conflict tracking, and graph queries.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- PackageGraphRequested
- PackageGraphValidated
- PackageGraphRejected
- PackageGraphMounted
- PackageGraphStateChanged
- PackageGraphFailed
- PackageGraphRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-006
- MAL-V03-B03-06-004
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-07-dependency-resolver-specification.md

Dependency Resolver Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-007

Related Blueprint Documents

- MAL-V03-B03-06-005
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for dependency resolution, compatibility checks, conflict decisions, lock records, and recovery in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for dependency resolution, compatibility checks, conflict decisions, lock records, and recovery.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DependencyResolverRequested
- DependencyResolverValidated
- DependencyResolverRejected
- DependencyResolverMounted
- DependencyResolverStateChanged
- DependencyResolverFailed
- DependencyResolverRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-007
- MAL-V03-B03-06-005
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-08-runtime-store-specification.md

Runtime Store Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-008

Related Blueprint Documents

- MAL-V03-B03-06-003
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime storage for package metadata, active mounts, locks, snapshots, indexes, and audit records in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime storage for package metadata, active mounts, locks, snapshots, indexes, and audit records.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- RuntimeStoreRequested
- RuntimeStoreValidated
- RuntimeStoreRejected
- RuntimeStoreMounted
- RuntimeStoreStateChanged
- RuntimeStoreFailed
- RuntimeStoreRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-008
- MAL-V03-B03-06-003
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-09-mount-unmount-hot-reload-specification.md

Mount Unmount Hot Reload Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-009

Related Blueprint Documents

- MAL-V03-B03-06-006
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for mount, unmount, remount, and hot reload behavior for governed DNA packages in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for mount, unmount, remount, and hot reload behavior for governed DNA packages.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- MountUnmountHotReloadRequested
- MountUnmountHotReloadValidated
- MountUnmountHotReloadRejected
- MountUnmountHotReloadMounted
- MountUnmountHotReloadStateChanged
- MountUnmountHotReloadFailed
- MountUnmountHotReloadRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-009
- MAL-V03-B03-06-006
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-10-universal-dna-extractor-specification.md

Universal DNA Extractor Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-010

Related Blueprint Documents

- MAL-V03-B03-06-008
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for extraction of reusable DNA signals from documents, workflows, agents, capabilities, and operating evidence in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for extraction of reusable DNA signals from documents, workflows, agents, capabilities, and operating evidence.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- UniversalDNAExtractorRequested
- UniversalDNAExtractorValidated
- UniversalDNAExtractorRejected
- UniversalDNAExtractorMounted
- UniversalDNAExtractorStateChanged
- UniversalDNAExtractorFailed
- UniversalDNAExtractorRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-010
- MAL-V03-B03-06-008
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-11-dna-validation-specification.md

DNA Validation Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-011

Related Blueprint Documents

- MAL-V03-B03-06-013
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for schema validation, policy validation, compatibility validation, and acceptance validation for DNA packages in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for schema validation, policy validation, compatibility validation, and acceptance validation for DNA packages.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNAValidationRequested
- DNAValidationValidated
- DNAValidationRejected
- DNAValidationMounted
- DNAValidationStateChanged
- DNAValidationFailed
- DNAValidationRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-011
- MAL-V03-B03-06-013
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-12-dna-versioning-specification.md

DNA Versioning Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-012

Related Blueprint Documents

- MAL-V03-B03-06-012
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for DNA package versions, semantic compatibility, supersession, rollback, and release lineage in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for DNA package versions, semantic compatibility, supersession, rollback, and release lineage.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNAVersioningRequested
- DNAVersioningValidated
- DNAVersioningRejected
- DNAVersioningMounted
- DNAVersioningStateChanged
- DNAVersioningFailed
- DNAVersioningRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-012
- MAL-V03-B03-06-012
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-13-dna-compatibility-specification.md

DNA Compatibility Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-013

Related Blueprint Documents

- MAL-V03-B03-06-012
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for compatibility matrices across runtimes, package versions, dependencies, tenants, and policy levels in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for compatibility matrices across runtimes, package versions, dependencies, tenants, and policy levels.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNACompatibilityRequested
- DNACompatibilityValidated
- DNACompatibilityRejected
- DNACompatibilityMounted
- DNACompatibilityStateChanged
- DNACompatibilityFailed
- DNACompatibilityRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-013
- MAL-V03-B03-06-012
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-14-dna-federation-specification.md

DNA Federation Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-014

Related Blueprint Documents

- MAL-V03-B03-06-009
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for federated DNA sharing, trust boundaries, import policies, provenance, and remote package references in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for federated DNA sharing, trust boundaries, import policies, provenance, and remote package references.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- DNAFederationRequested
- DNAFederationValidated
- DNAFederationRejected
- DNAFederationMounted
- DNAFederationStateChanged
- DNAFederationFailed
- DNAFederationRecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-014
- MAL-V03-B03-06-009
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.

spec-06-dna-15-export-evolved-dna-specification.md

Export Evolved DNA Specification

Version: v26 draft

Status: Draft

Specification ID: MAL-V06-DNA-SPEC-015

Related Blueprint Documents

- MAL-V03-B03-06-020
- MAL-V03-B03-06-README
- MAL-V03-B03-06-001
- MAL-V03-B03-01-008
- MAL-V05-STD-011
- MAL-V05-STD-012

Purpose

Define executable requirements for export of evolved DNA packages with review, provenance, compatibility, security, and release evidence in the Mariam DNA Operating System.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for export of evolved DNA packages with review, provenance, compatibility, security, and release evidence.

Responsibilities

- Convert the DNA Operating System blueprint into implementation-ready subsystem requirements.
- Define package lifecycle behavior without allowing unreviewed package mutation or unsafe runtime side effects.
- Preserve provenance, compatibility, governance, security classification, and release evidence for DNA packages.
- Integrate with Enterprise Core configuration, identity access, runtime, event bus, storage, security, and observability.

Functional Requirements

- The subsystem MUST validate DNA package metadata, schema version, source provenance, trust level, and tenant scope before activation.
- The subsystem MUST reject packages or operations that violate policy, dependency, compatibility, version, or security requirements.
- The subsystem MUST emit canonical events for import, validation, mount, unmount, reload, export, rejection, failure, and recovery.
- The subsystem MUST maintain deterministic state transitions for draft, staged, validated, mounted, active, degraded, deprecated, archived, and rejected package states.
- The subsystem MUST support dry-run validation before package activation.

- The subsystem MUST preserve enough evidence to reproduce package acceptance decisions.

Non-Functional Requirements

- Package operations SHOULD be idempotent where repeated commands may occur after retries.
- Derived indexes, graphs, and runtime views MUST be rebuildable from canonical package and audit records.
- Protected package operations MUST include authorization, tenant context, correlation ID, and audit trail.
- Runtime package operations SHOULD degrade safely instead of corrupting active package state.
- Package validation and graph operations SHOULD provide deterministic output for identical inputs.

Inputs

- DNA package archive, MDP metadata, Git reference, ZIP archive, exported package, or extracted DNA candidate.
- Caller identity, tenant context, role, policy, trust level, and requested operation.
- Runtime compatibility matrix, dependency constraints, package graph state, and configuration.
- Source provenance, checksum, signature, version, and classification metadata.

Outputs

- Validated or rejected package records.
- Mounted package runtime objects, dependency locks, graph edges, version records, and compatibility decisions.
- Events, metrics, logs, traces, and audit records.
- Exportable package artifact or evolved DNA package where applicable.

Public Interfaces

- POST /dna/packages/import
- POST /dna/packages/validate
- POST /dna/packages/mount
- POST /dna/packages/unmount
- POST /dna/packages/export
- GET /dna/packages/{package_id}
- GET /dna/graph

Internal Interfaces

- Enterprise Core identity access guard.
- Configuration provider and secret provider where applicable.
- Runtime manager for activation and reload operations.
- Event bus for DNA lifecycle events.
- Core storage for package records, graph records, locks, and audit history.
- Observability pipeline for metrics, logs, traces, and health signals.

Events

- ExportEvolvedDNARequested
- ExportEvolvedDNAValidated
- ExportEvolvedDNARejected
- ExportEvolvedDNAMounted
- ExportEvolvedDNAStateChanged
- ExportEvolvedDNAFailed
- ExportEvolvedDNARecovered

Storage Requirements

- Store canonical package metadata, checksums, source references, version, compatibility, and trust records.
- Store graph nodes, dependency edges, locks, mount records, validation reports, and export evidence.
- Store immutable audit references for protected operations.
- Store derived indexes separately from canonical package records so they can be rebuilt.

Security Requirements

- Enforce least privilege for import, validation, mount, hot reload, federation, export, and deletion.
- Deny packages from untrusted or unknown sources unless a policy exception is approved.
- Redact secrets from packages, metadata, logs, diagnostics, exports, and validation reports.
- Validate package paths and archive contents to prevent traversal, overwrite, or unauthorized file access.
- Audit privileged operations, denied operations, policy exceptions, and federation imports.

Observability Requirements

- Emit metrics for validation time, mount time, dependency resolution time, import failures, rejected packages, active package count, and rollback count.
- Emit structured logs with package ID, version, tenant, operation, result, and correlation ID.
- Emit traces around import, validation, dependency resolution, mount, export, and recovery paths.
- Provide health signals for package graph integrity, runtime store consistency, and mounted package readiness.

Failure Modes

- Invalid package schema, missing metadata, checksum mismatch, incompatible version, or dependency conflict.
- Unauthorized caller, missing tenant context, denied source, or policy exception required.
- Archive extraction failure, Git reference failure, storage write failure, event publication failure, or runtime mount failure.
- Hot reload leaves dependent services stale or degraded.
- Federation import conflicts with local package authority.

Recovery Rules

- Roll back partially mounted packages to the last accepted runtime state.
- Rebuild package graph and indexes from canonical records after storage or index failure.
- Quarantine failed imports until review or explicit deletion.
- Retry idempotent archive, graph, and storage operations with bounded attempts.

- Require operator review for repeated validation, mount, export, or federation failures.

Test Requirements

- Unit tests for schema validation, compatibility decisions, dependency resolution, and state transitions.
- Integration tests across identity access, runtime manager, event bus, storage, configuration, and observability.
- Security tests for archive traversal, untrusted sources, tenant isolation, secret redaction, and denied operations.
- Recovery tests for partial mount, hot reload failure, graph rebuild, and import rollback.
- Acceptance tests proving required events, records, metrics, and audit evidence exist.

Acceptance Criteria

- The specification is detailed enough to implement without relying on undocumented package behavior.
- Functional and non-functional requirements are measurable and testable.
- Security, observability, failure, and recovery behavior are explicit.
- DNA package authority, provenance, validation, compatibility, and lifecycle are traceable.

Implementation Notes

- Begin with read-only import and validation before enabling mount, hot reload, federation, or export.
- Keep package parsing isolated from runtime activation.
- Use structured metadata and typed records instead of free-form package state.
- Treat every compatibility bypass as a governance exception with an expiry.

Traceability

- MAL-V06-DNA-SPEC-015
- MAL-V03-B03-06-020
- MAL-V03-B03-06-README
- MAL-V05-STD-011
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v26 draft	2026-06-29	Draft	Initial DNA Operating System specification.